



Geovisualización inteligente

Cuando trabajamos con visualización de datos geoespaciales, crear mapas que permitan la interacción del usuario es una forma poderosa de hacer más accesible y dinámico el análisis espacial. Con Python, una herramienta clave para esto es **Folium**, una biblioteca que facilita la creación de mapas interactivos. Además, gracias al método `explore()` de GeoPandas, podemos generar mapas interactivos de manera casi inmediata, directamente a partir de

nuestros datos espaciales. Esto nos permite **visualizar, desplazarnos y consultar atributos de forma intuitiva, sin necesidad de programar desde cero toda la interacción.**

Estas herramientas no sólo facilitan la creación de mapas interactivos, sino que forman parte de una tendencia más amplia: la **geovisualización inteligente.**

La geovisualización inteligente combina el poder de los datos geoespaciales con tecnologías interactivas y de inteligencia artificial para **mejorar la forma en que interpretamos, analizamos y presentamos información en un mapa.** A diferencia de las visualizaciones estáticas tradicionales, esta nueva forma de representar datos permite al usuario interactuar con el mapa, explorar diferentes niveles de información y obtener respuestas más precisas a partir del contexto geográfico.

Hoy en día, herramientas como los mapas interactivos permiten filtrar, agrupar o resaltar datos directamente en la interfaz, facilitando la toma de decisiones en tiempo real. Además, con el apoyo de la inteligencia artificial, es posible automatizar la **identificación de patrones, sugerir visualizaciones relevantes o incluso interpretar los datos para generar conclusiones preliminares.** Este enfoque no sólo hace más accesible la información geoespacial, sino que también abre nuevas posibilidades para su análisis y aplicación en proyectos reales.

| Mapas interactivos

Revisaremos un primer ejemplo de cómo crear un mapa interactivo utilizando la librería **Folium**, que aprovecha las ventajas de la manipulación de datos con **Python** y la construcción de mapas con la biblioteca **leaflet.js**. Esto nos permite trabajar datos en un **GeoDataFrame** y visualizarlos de manera sencilla usando **Folium**.

Veamos un ejemplo con la información de los estados de la República Mexicana, que ya usamos en temas previos.

Lo primero que tenemos que hacer es importar la librería **Folium**. Si estás trabajando en una libreta de Google Colab, no es necesario instalar nada; sólo basta con importarla:

```
import folium
```

Como siguiente paso, creamos un mapa, que en este caso llamaremos `my_map`. Los parámetros que usaremos serán una lista con dos valores numéricos (latitud y longitud), que no necesariamente son enteros, asignados a `location`, junto con otro valor numérico para el atributo `zoom_start`, que define el nivel de *zoom* aplicado a ese punto en el mapa.

```
my_map = folium.Map(location=[24, -97], zoom_start = 5.4)
# las coordenadas mostradas son una aproximación del
# centro de la república mexicana,
# el zoom inicial de 5.4 es un zoom que nos permite ver
# todo el país
```

Si ejecutamos este código, podemos ver que de manera muy sencilla se genera un mapa centrado en las coordenadas especificadas con el nivel de *zoom* definido. En este mapa podemos movernos simplemente arrastrándolo con el *mouse*, así como ajustando el nivel de *zoom* aplicado.

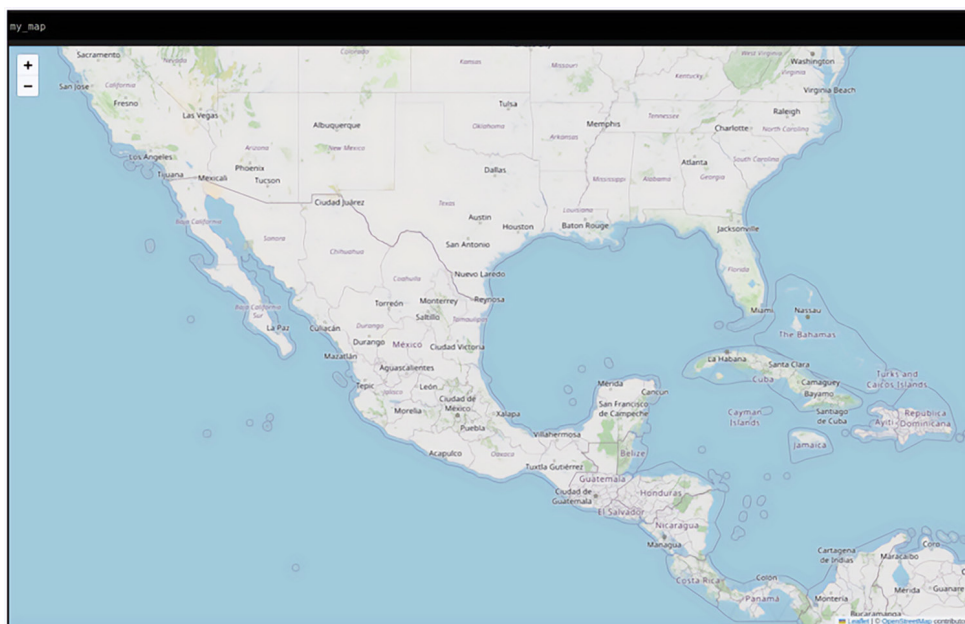


Figura 1. Mapa resultado de la ejecución del código.

Bien, ya creamos un primer mapa. Ahora mostremos información en él, por ejemplo, las capitales de cada uno de los estados de la República Mexicana. Para esto, retomemos el archivo que contiene esta información y que utilizamos en lecciones anteriores. En la lección 3: Mapas, te mostramos cómo cargarlo en Colab.

Primero, importemos las bibliotecas *pandas* y *GeoPandas*:

```
import pandas as pd
import geopandas as gpd
```

Luego, carguemos el archivo CSV que contiene esta información en un *DataFrame*:

```
df = pd.read_csv('mexican-capitals.csv')
```

Ahora, creamos el GeoDataFrame utilizando la información de longitud y latitud para generar la geometría:

```
capitales = gpd.GeoDataFrame(df, geometry=gpd.points_
from_xy(df.Long, df.Lat))
```

Una vez que ya tenemos nuestro GeoDataFrame construido adecuadamente, podemos recorrer cada uno de sus renglones para tomar la latitud y longitud de cada estado y pasarlas como primer parámetro a `Marker`, una clase de la biblioteca **Folium** que nos permite poner marcadores en el mapa.

Como segundo parámetro, pasamos el valor que queremos que se despliegue cuando el usuario dé clic en ese marcador; este valor lo asignamos al atributo `popup`. Finalmente, este marcador recién creado lo asignamos a `my_map`, el mapa que habíamos creado previamente.



Aquí te dejamos un fragmento del código y el resultado de la ejecución:

```
# iteramos cada uno de los renglones del DataFrame
"capitales" para
# 1. extraer la latitud y la longitud y pasarlos como
primer parámetro al método Marker
# 2. extraer el Estado y la Capital, y los concatenamos
para mostrarlos cuando se haga click en ese marcador
# 3. finalmente agregamos el marcador al mapa "my_map"

for i in range(len(capitales)):
    folium.Marker(capitales.loc[i][['Lat','Long']], popup
= capitales.loc[i]['Estado'] + " - " + capitales.loc[i]
['Capital']).add_to(my_map)
```

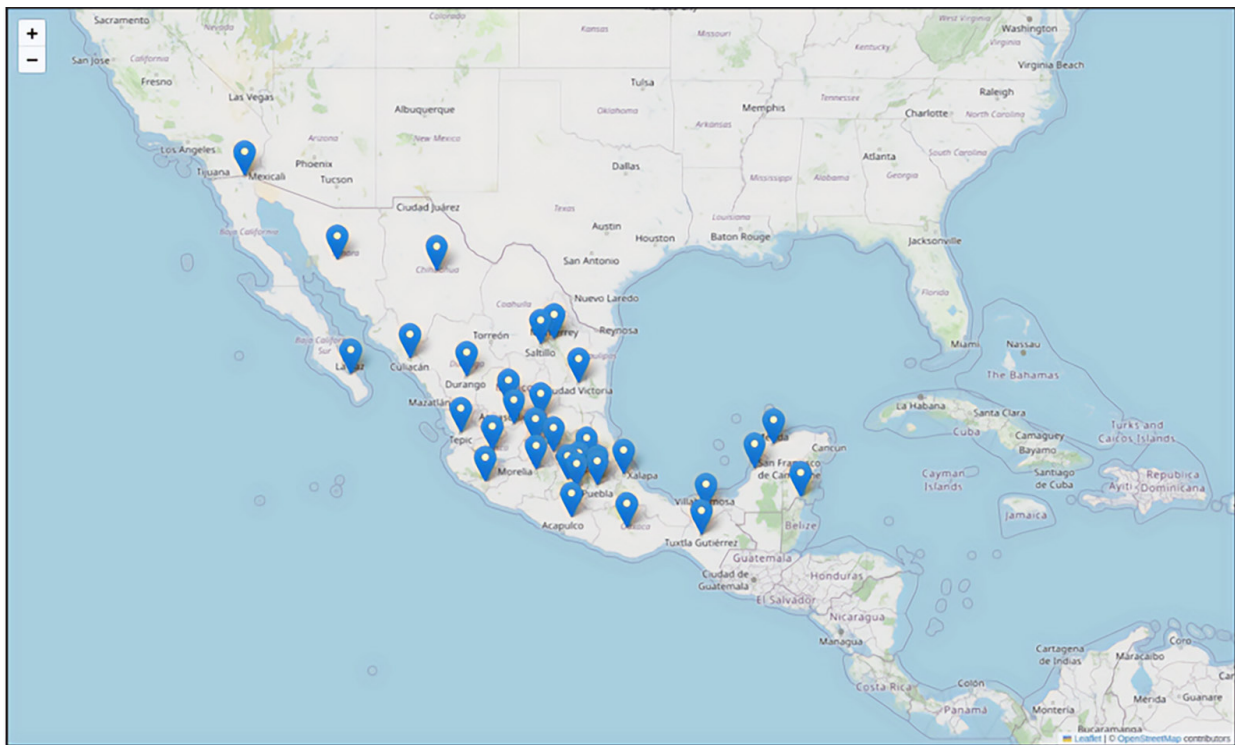


Figura 2. Mapa con marcadores de estados.

Al dar clic en algún marcador, aparece la información del Estado y la Capital, tal como lo habíamos programado.

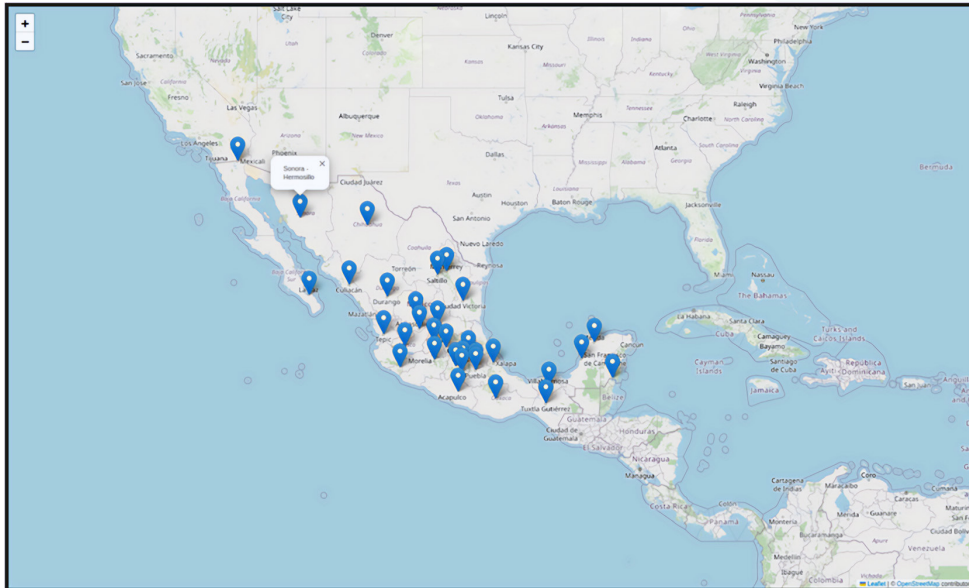


Figura 3. Marcadores activados con información del estado y capital.

De esta manera, podemos crear mapas con marcadores utilizando las coordenadas de los puntos, y hacer que, cuando el usuario dé clic en el marcador, aparezca la información que queramos mostrar.

En Folium, podemos utilizar otros métodos como `folium.PolyLine`, que se usa para dibujar líneas que conectan puntos en un mapa, y `folium.Polygon`, que permite dibujar formas cerradas, como áreas o regiones, usando un conjunto de puntos conectados. Ambos son elementos muy útiles para visualizar datos geográficos en mapas interactivos creados con Folium.

Para más detalles, puedes revisar la documentación de Folium que encuentras en la sección de material adicional de esta lectura.

Pasemos ahora a otro ejemplo que puede ser más vistoso y un poco más elaborado, utilizando otra biblioteca que nos facilita el trabajo: el **método `explore`** de `GeoDataFrame`. Pongamos manos a la obra retomando otro ejemplo que ya habíamos trabajado: la información de los estados de la República Mexicana.

Si recuerdas, en la **lección 3: Mapas**, utilizábamos varios archivos llamados `mexican-states`. Retomemos este ejemplo para mostrar la funcionalidad del método `explore()`.

Recuerda que, para trabajar con este ejemplo, a pesar de que sólo se carga un el archivo (`mexican-states.shp`), es necesario tener todos los archivos `mexican-states` con sus diferentes extensiones en tu directorio de trabajo.

Primero, creamos el `DataFrame` `mex` con la información de los estados leyendo el archivo `'mexican-states.shp'`.

```
mex = gpd.read_file('mexican-states.shp' )
```

Algo importante a tomar en cuenta es que, para poder trabajar con el método `explore()`, necesitamos importar las bibliotecas `matplotlib` y `mapclassify`. Para ello, es necesario instalar previamente `mapclassify`.

```
!pip install mapclassify
```

```
# Importamos bibliotecas necesarias para usar explore()
import matplotlib
import mapclassify
```

Ahora sí, podemos llamar al método `explore` del `GeoDataFrame` **mex**

```
mex.explore()
```

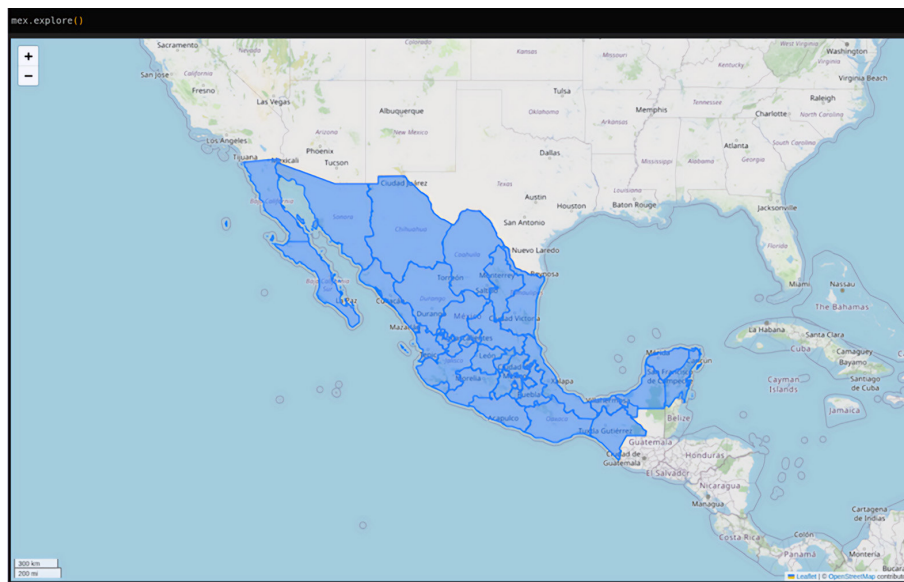


Figura 4. Mapa con geometrías de cada estado.

Podemos ver que, de manera muy fácil, generamos un mapa con las geometrías de cada estado, en el que, al pasar el *mouse* por encima de cada uno de ellos, se nos muestra toda la información de esa fila en el `GeoDataFrame`.

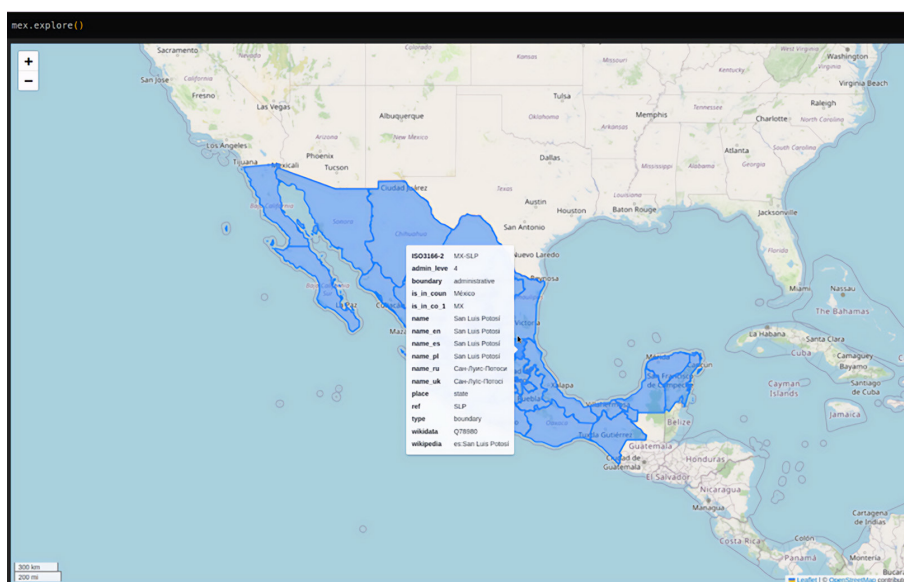


Figura 5. Mapa activado con la información de cada estado.

Muy probablemente no queremos ver todas las columnas de cada fila en el GeoDataFrame, así que podemos respaldar nuestro GeoDataFrame original y trabajar sólo con un subconjunto de columnas. Reemplazamos el nombre de la columna `name` por `Estado`, para que tenga más sentido al momento de visualizar el mapa:

```
mex.columns

mex2 = mex
mex = mex[['ISO3166-2', 'name', 'place', 'ref',
          'wikidata', 'wikipedia', 'geometry']]
mex = mex.rename(columns={"name": "Estado"})
```

Ahora usaremos nuevamente el método `explore()`, pero esta vez con algunos parámetros para ver un uso más avanzado:

- Usaremos el atributo `column` para indicar cuál es la columna que determinará el mapa generado.
- Usaremos el atributo `tooltip` para indicar qué valor(es) queremos desplegar cuando el puntero del *mouse* pase por esa región del mapa.
- Usaremos el atributo `popup` con el valor `True` para que, al hacer clic en esa región (estado), se despliegue la información de todas las columnas en nuestro GeoDataFrame.
- Usaremos el atributo `cmap="Set1"` para aplicar el conjunto de colores *Set1* de la biblioteca `matplotlib`.
- Finalmente, usaremos el atributo `style_kwds=dict(color="black")` para indicar que se utilice el color negro como contorno de cada una de las filas del GeoDataFrame.

El código y el resultado quedarían de la siguiente manera:

```
mex.explore(  
    column="Estado", # Hacemos un mapa de colores basado  
    en la columna "Estado"  
    tooltip="Estado", # Mostramos el valor de la columna  
    "Estado" cuando el puntero del mouse pase sobre esa  
    región  
    popup=True, # Muestra todos los valores del  
    GeoDataFrame cuando ese haga clic en esa región  
    cmap="Set1", # utiliza el mapa de colores "Set1" de la  
    biblioteca matplotlib  
    style_kwds=dict(color="black"), # pone en color negro  
    los contornos  
)
```

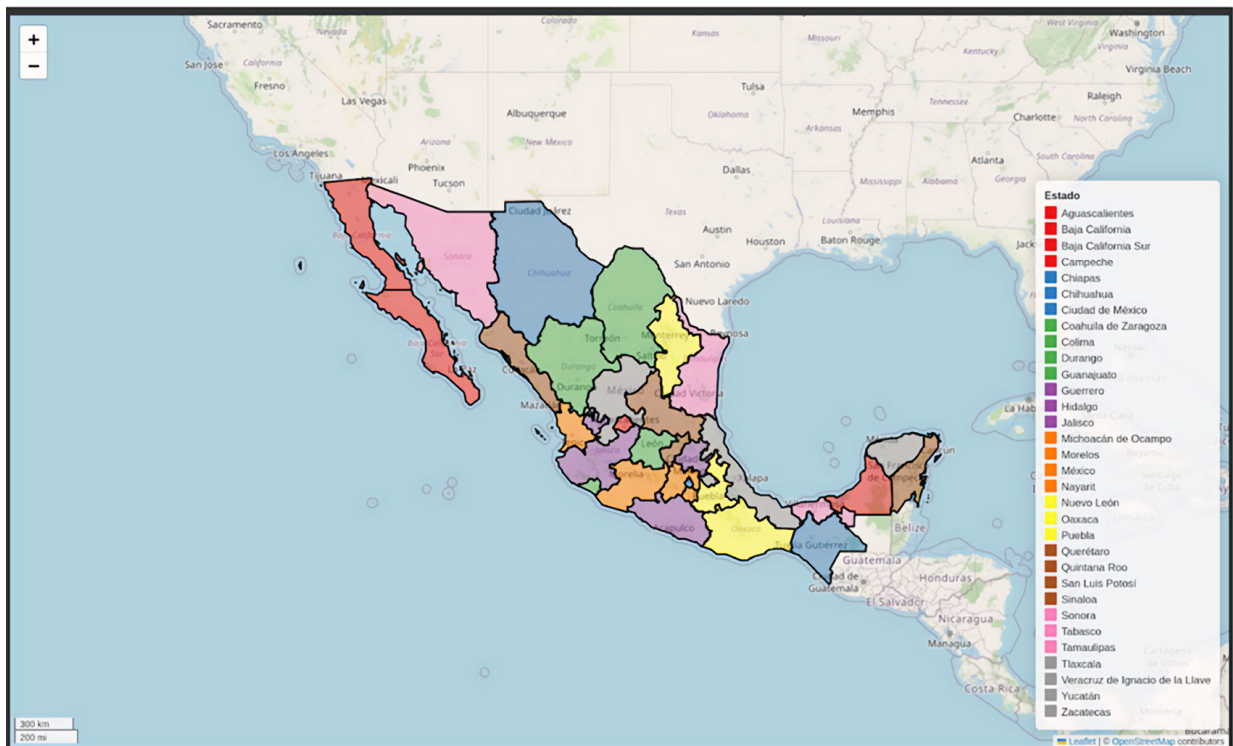


Figura 6. Mapa con atributos avanzados usando explore ().

Al pasar el puntero del *mouse* sobre un estado, se despliega el nombre de este.

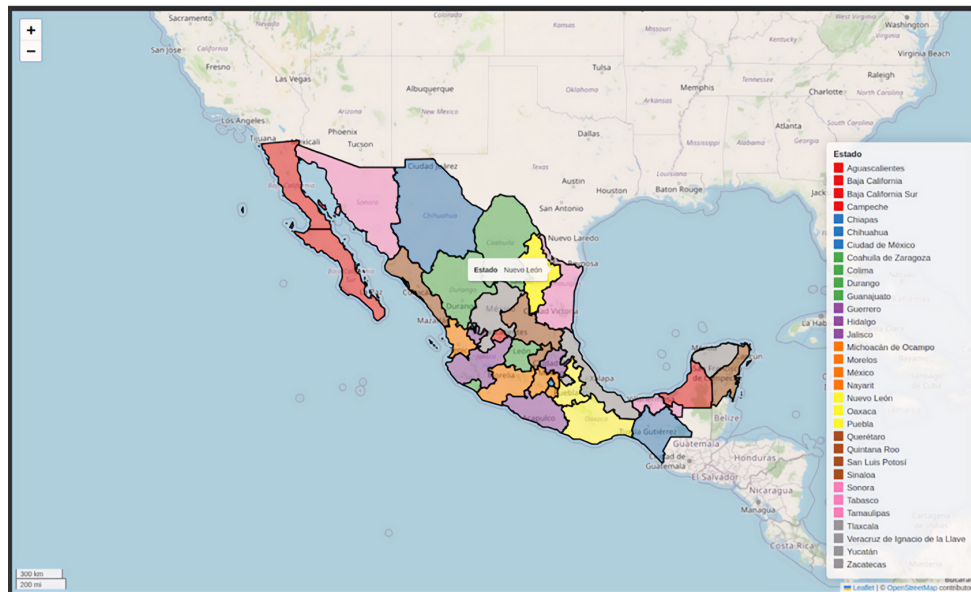


Figura 7. Mapa con atributos avanzados, al pasar el *mouse*.

Al dar clic en algún estado, nos muestra la información de cada una de las columnas de esa fila del GeoDataFrame.

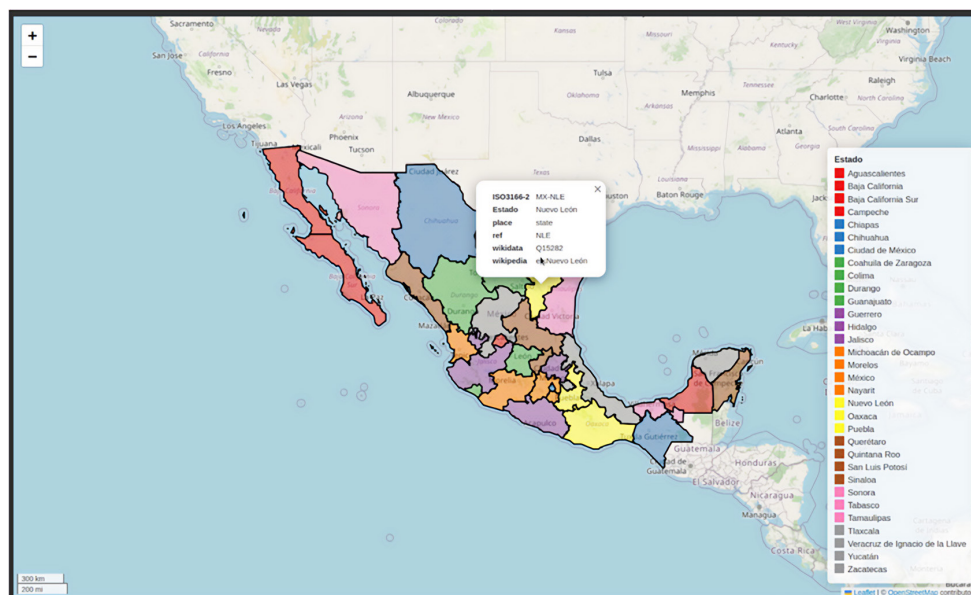


Figura 8. Mapa con atributos avanzados, al dar clic.

Hasta este momento, hemos mostrado información de los estados, pero probablemente queramos mostrar algún tipo de marcador sobre ubicaciones específicas. Por eso, retomaremos nuestro GeoDataFrame que contiene información de las capitales y la mostraremos en el mismo mapa.

Lo primero que tenemos que hacer es crear un mapa con el método `explore()` y guardarlo en una variable, para que después podamos pasar esa variable como parámetro a otro mapa que graficaremos:

```
m = mex.explore(  
    column="Estado", # Hacemos un mapa de colores basado  
    en la columna "Estado"  
    tooltip="Estado", # Mostramos el valor de la columna  
    "Estado" cuando el puntero del mouse pase sobre esa  
    región  
    popup=True, # Muestra todos los valores del  
    GeoDataFrame cuando se haga clic en esa región  
    cmap="Set1", # Utiliza el mapa de colores "Set1" de la  
    biblioteca matplotlib  
    style_kwds=dict(color="black"), # Pone en color negro  
    los contornos  
)
```

Este fragmento de código ya lo habíamos revisado, sólo que ahora lo asignamos a la variable `m`.

Posteriormente, creamos el mapa de las capitales, pero pasamos como primer parámetro `m=m` para indicar que genere este mapa sobre el que ya se había creado. Asignamos `color="blue"` para indicar que queremos que los marcadores de las capitales se desplieguen en azul, y finalmente asignamos `tooltip=['Estado', 'Capital']` para indicar que cuando el puntero del *mouse* pase sobre ese punto, se muestren el Estado y su Capital.

```
capitales.explore(  
    m=m, # Utiliza el mismo mapa  
    color="blue", # Define el color del marcador de las  
    capitales en azul  
    tooltip=['Estado', 'Capital'] # Mostramos el valor de  
    "Estado" y "Capital" cuando el puntero del mouse pase  
    sobre ese punto  
)
```

Posteriormente, de manera opcional, podemos agregar un control de capas al mapa. Este es el pequeño icono que aparece arriba a la derecha y que nos permite activar y desactivar cada una de las capas que estamos utilizando en nuestro mapa.

Y finalmente mostramos el mapa.



El código completo de esta parte queda de la siguiente manera:

```
m = mex.explore(
    column="Estado", # Hacemos un mapa de colores basado
    en la columna "Estado"
    tooltip="Estado", # Mostramos el valor de la columna
    "Estado" cuando el puntero del mouse pase sobre esa
    región
    popup=True, # Muestra todos los valores del
    GeoDataFrame cuando se haga clic en esa región
    cmap="Set1", # Utiliza el mapa de colores "Set1" de la
    biblioteca matplotlib
    style_kwds=dict(color="black"), # Pone en color negro
    los contornos
)

capitales.explore(
    m=m, # Utiliza el mismo mapa
    color="blue", # Define el color del marcador de las
    capitales en azul
    tooltip=['Estado', 'Capital'] # Mostramos el valor de
    "Estado" y "Capital" cuando el puntero del mouse pase
    sobre ese punto
)

folium.LayerControl().add_to(m) # Agregamos un control
de capas al mapa

m # Mostramos el mapa
```

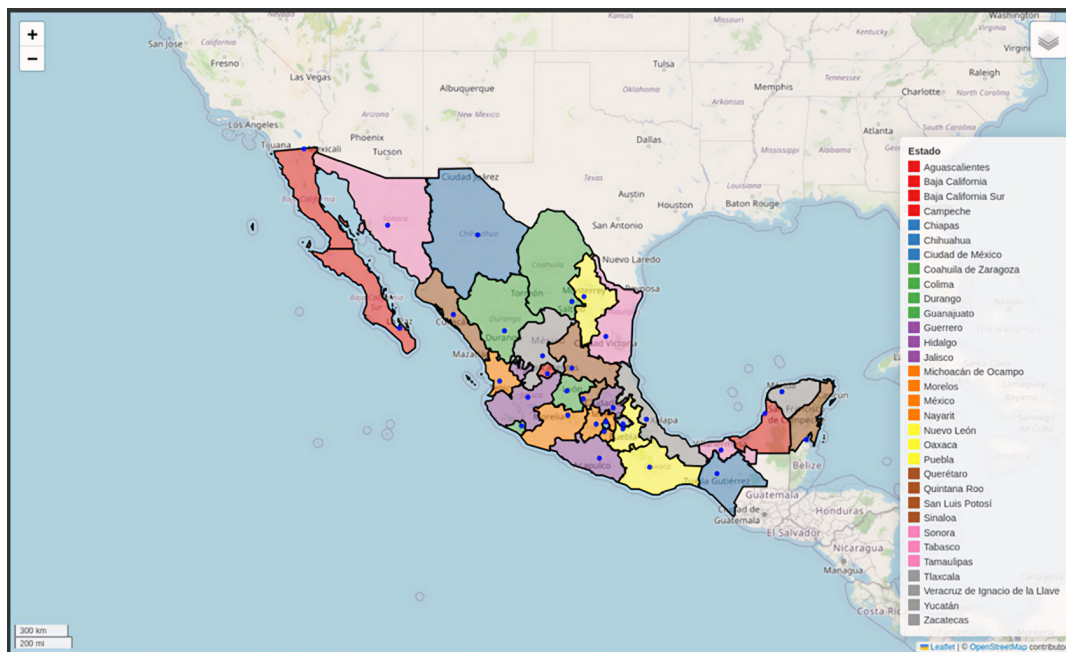


Figura 9. Control de capas.

El mapa se muestra con las dos capas.

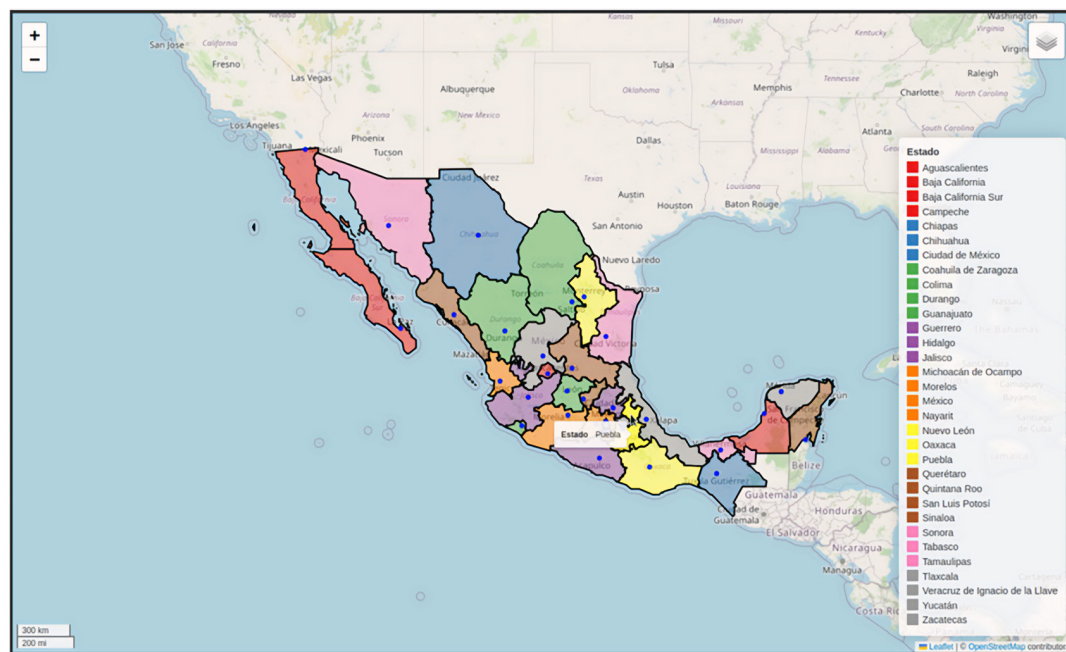


Figura 10. Mapa con control de capas activado al pasar el mouse.

Al pasar el *mouse* por un estado, se muestra el nombre del estado.

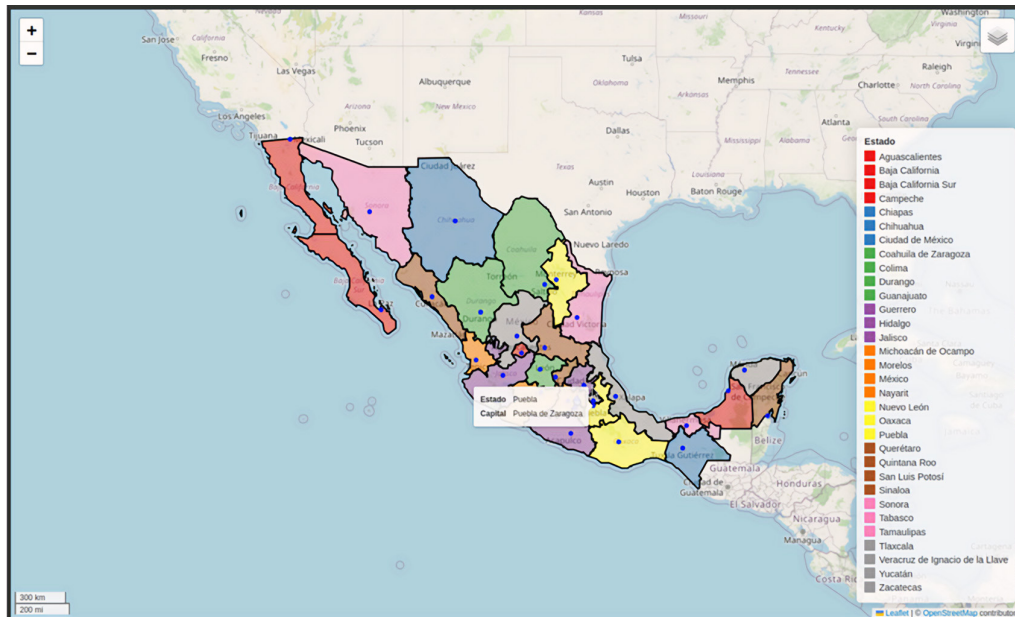


Figura 11. Información del estado y capital al pasar el *mouse*.

Al pasar el ratón por el marcador de la capital, se muestra la información del estado y la capital.



Figura 12. Información del estado al dar clic.

Al hacer clic en algún estado, se muestra toda la información de ese estado.



Figura 13. Ejemplo de capas activadas/desactivadas al dar clic.

Dando clic en el icono de las capas podemos activar y desactivar cada una de ellas.

Los mapas interactivos permiten explorar visualmente patrones y relaciones en los datos geoespaciales. Sin embargo, cuando los datos crecen en volumen y complejidad, la interpretación asistida por IA se vuelve clave, aprendamos más a continuación.

| Interpretación asistida por IA

La interpretación de datos geoespaciales puede volverse compleja cuando trabajamos con grandes volúmenes de información o múltiples capas de datos superpuestas. Para facilitar esta tarea, hoy en día podemos apoyarnos en herramientas de inteligencia artificial integradas en plataformas como Gemini de Google, que está integrada en el IDE de Google Colab que hemos venido utilizando. Estas herramientas permiten analizar, resumir y generar interpretaciones automáticas sobre mapas y datos geográficos, lo cual agiliza los procesos de análisis y mejora la toma de decisiones.

En este subtema aprenderás a utilizar los asistentes de IA en Google Colab para interpretar los resultados de operaciones con GeoDataFrames. Estas funciones pueden ayudarte a **entender patrones espaciales, identificar regiones con ciertas características, generar resúmenes de zonas geográficas y formular hipótesis a partir de visualizaciones interactivas.**

Exploraremos cómo integrar la visualización geoespacial con la capacidad de razonamiento de los asistentes de IA, combinando el poder de bibliotecas como **GeoPandas** con la asistencia contextual de modelos de lenguaje. Esta fusión entre datos y comprensión automática abre nuevas posibilidades para quienes trabajan con mapas y necesitan generar conclusiones claras y accionables rápidamente.

En el siguiente ejemplo guiado, trabajaremos con el mismo contexto de los estados, capitales y zonas de la República Mexicana.

Imagina que el gobierno federal quiere identificar regiones del país que podrían estar desatendidas en cuanto a servicios educativos. Para ello, se ha definido una zona de influencia alrededor de cada capital estatal (por ejemplo, un radio de 50 km). Se busca determinar qué estados tienen la menor cobertura de estas zonas, y por lo tanto, podrían requerir atención prioritaria.

Te guiaremos paso a paso para que pruebes la herramienta de Gemini que está integrada en Google Colab. Por favor, revisa y copia el código disponible en cada paso, y ajústalo para que funcione de acuerdo con la localización de los archivos en tu Drive.

1 Monta en Google Drive

```
from google.colab import drive
drive.mount('/content/gdrive')
```



2 Carga los datos

```
import pandas as pd
import geopandas as gpd

# Cargar los estados y las capitales
estados = gpd.read_file('/content/gdrive/MyDrive/mexican-
states.shp' )
df = pd.read_csv('/content/gdrive/MyDrive/mexican-
capitals.csv' )
capitales = gpd.GeoDataFrame(df, geometry=gpd.points_
from_xy(df.Long, df.Lat))

# Asegurarse de que ambos usen el mismo sistema de
referencia
capitales = capitales.set_crs(estados.crs)
estados = estados.to_crs(epsg=3395)
capitales = capitales.to_crs(epsg=3395)
```

3 Crea zonas de influencia alrededor de cada capital

```
# Crear círculos de 50 km alrededor de las capitales
capitales['Zona_Influencia'] = capitales.geometry.
buffer(50000)

# Convertir los círculos en un GeoDataFrame separado
zonas_influencia = gpd.GeoDataFrame(capitales[['Capital']],
geometry=capitales['Zona_Influencia'])
```

4 Haz una intersección para ver qué parte del estado está cubierta

```
# Intersección entre zonas de influencia y estados
estados_con_cobertura = gpd.overlay(estados, zonas_
influencia, how='intersection', keep_geom_type=False)
```

5 Visualiza los resultados

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(12, 8))
estados.plot(ax=ax, color='lightgrey', edgecolor='black')
zonas_influencia.plot(ax=ax, color='skyblue', alpha=0.4)
estados_con_cobertura.plot(ax=ax, color='green',
alpha=0.6)
plt.title("Cobertura educativa estimada por proximidad a capitales")
plt.show()
```

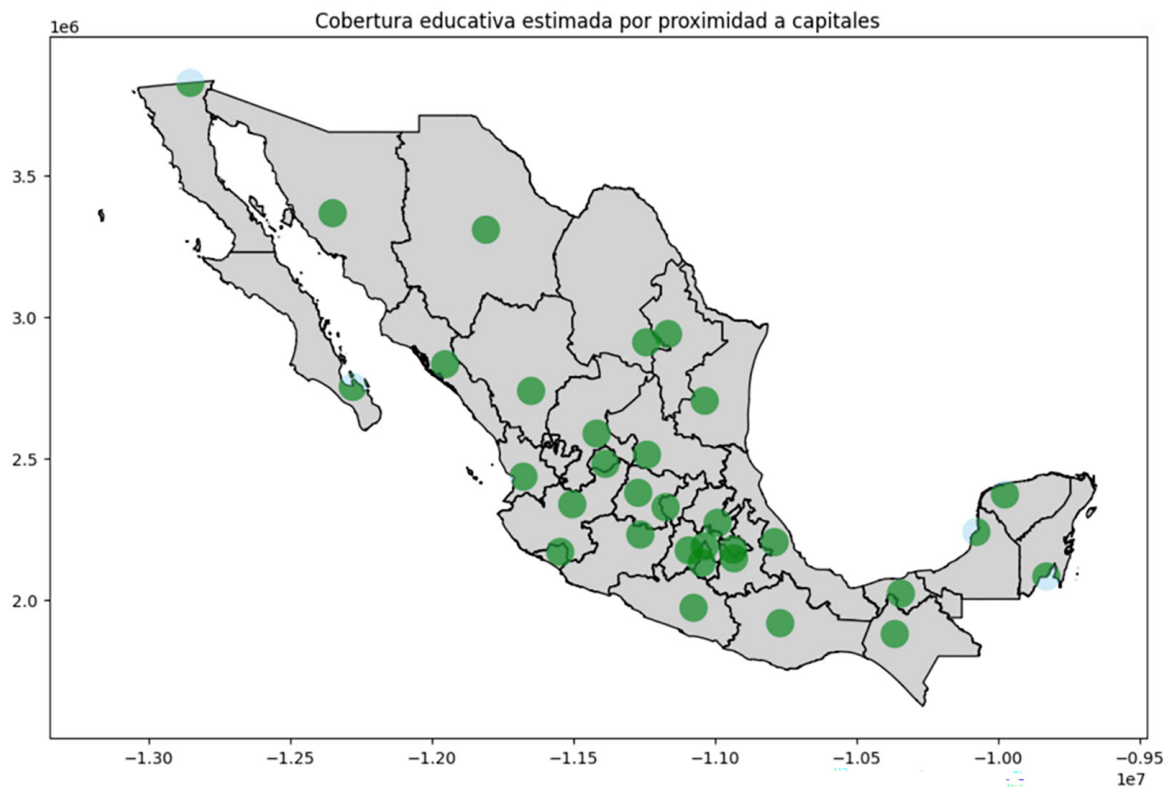


Figura 14. Mapa con zonas de afluencia.

6 Analiza la interpretación asistida por IA

Existen varias maneras de iniciar la herramienta de Gemini, la primera es utilizar el botón que se encuentra en la parte superior derecha de la pantalla. Esto abrirá el *frame* de Gemini del lado derecho de la pantalla. Si es la primera vez que lo utilizas es probable que te pida permiso para poder ejecutarla. En esta ventana, en la parte central van a aparecer algunos ejemplos de *prompts* que te podrían ser de utilidad, o, puedes escribir en el campo de texto de la parte inferior tus preguntas.

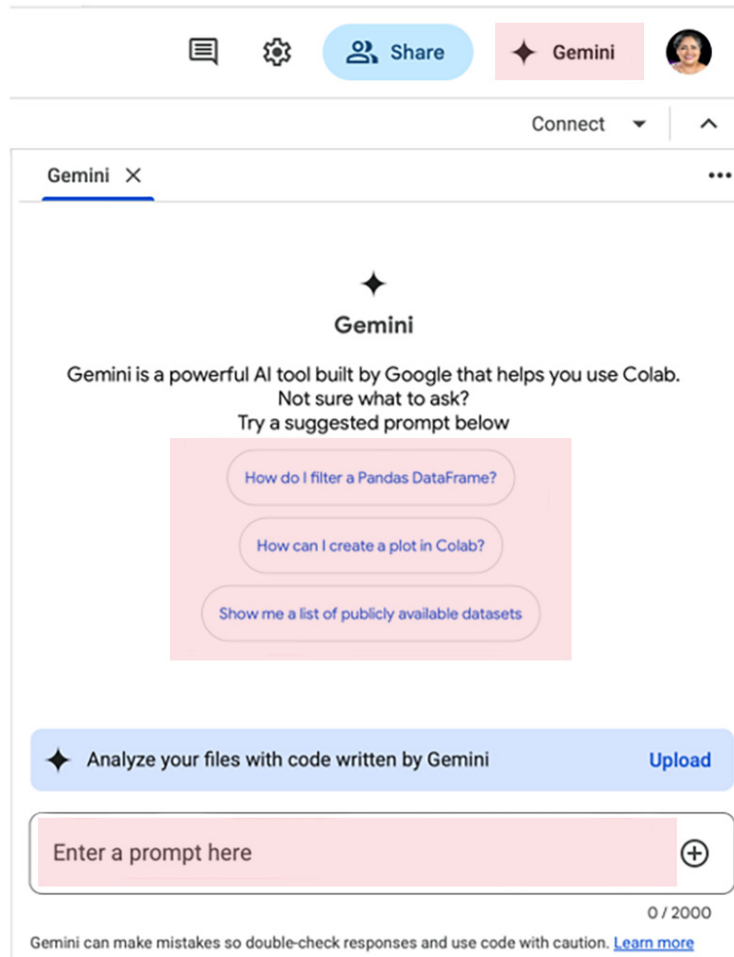


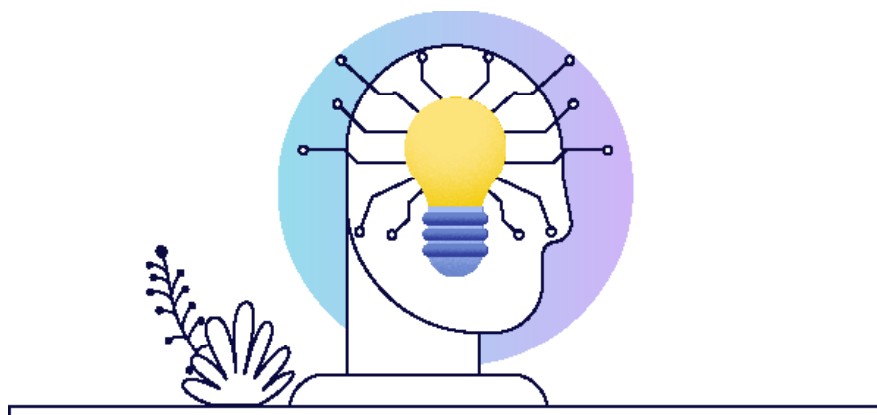
Figura 15. Interpretación asistida por IA.

La IA te puede ayudar a realizar diversas acciones. Sin embargo, es importante recordar que puede cometer errores, por lo que siempre debes asegurarte de que sus respuestas sean coherentes y adecuadas según tus necesidades:

- Generar una descripción textual del mapa.
- Identificar visualmente qué regiones están menos cubiertas.
- Proponer acciones o hipótesis.
- Redactar conclusiones o reportes automáticos.

Vamos a intentar con el siguiente *prompt*:

“Considera que el gobierno federal quiere identificar regiones del país que podrían estar desatendidas en cuanto a servicios educativos. Para ello, se ha definido una zona de influencia alrededor de cada capital estatal (por ejemplo, un radio de 50 km). Se busca determinar qué estados tienen la menor cobertura de estas zonas y, por lo tanto, podrían requerir atención prioritaria. El mapa generado muestra este resultado, pero me gustaría que me proporciones una descripción textual del mapa y propongamos acciones o hipótesis dentro de este contexto.”





El resultado generado es el siguiente.

Descripción del mapa

El mapa muestra los estados de México, con las capitales de cada estado marcadas. Alrededor de cada capital, se ha dibujado un círculo que representa una zona de influencia (por ejemplo, de 50 km de radio). Las áreas donde estas zonas de influencia se superponen con los estados se muestran en verde, lo que indica la cobertura estimada de servicios educativos. Las áreas de los estados que no están cubiertas por estas zonas de influencia se muestran en gris claro, sugiriendo posibles áreas desatendidas.

Acciones o hipótesis

Basándonos en la información del mapa, podemos proponer las siguientes acciones o hipótesis:

- **Identificar los estados con menor cobertura:** Observando el mapa, podemos identificar los estados con menor cobertura de las zonas de influencia, es decir, aquellos con mayor área en gris claro. Estos estados podrían requerir atención prioritaria en la provisión de servicios educativos.
- **Analizar la densidad poblacional en áreas desatendidas:** Se podría superponer al mapa información sobre la densidad poblacional para determinar si las áreas desatendidas tienen una alta concentración de población. Esto ayudaría a priorizar las zonas que requieren mayor inversión en infraestructura educativa.
- **Considerar factores geográficos:** Las áreas desatendidas podrían estar ubicadas en zonas de difícil acceso o con características geográficas que dificultan la provisión de servicios. Es importante tener en cuenta estos factores al planificar la expansión de la cobertura educativa.

- **Evaluar la oferta educativa existente:** Aunque un área no esté cubierta por la zona de influencia de una capital, podría tener otras instituciones educativas que brinden servicios a la población. Es necesario evaluar la oferta educativa existente en las áreas desatendidas para determinar las necesidades reales.
- **Planificar la construcción de nuevas instituciones educativas:** Basándose en el análisis de la cobertura, la densidad poblacional y la oferta educativa existente, se puede planificar la construcción de nuevas instituciones educativas en las áreas desatendidas para asegurar un acceso equitativo a la educación.
- **Fortalecer los programas de educación a distancia:** En zonas de difícil acceso, se podrían fortalecer los programas de educación a distancia para brindar oportunidades educativas a la población que reside en áreas desatendidas.

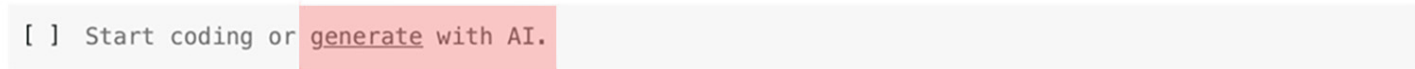
Recuerda que esta interpretación se basa en una simplificación de la realidad, y es necesario considerar otros factores para una toma de decisiones informada.

Antes de continuar, te sugerimos intentar con otros *prompts* que apoyen el análisis de la información obtenida en el mapa. Una vez que termines, reflexiona sobre las siguientes preguntas:

- ¿Qué zonas observas con menor cobertura?
- ¿Qué factores podrían influir en esta falta de cobertura?
- ¿La IA propuso observaciones que tú no habías considerado?

Una segunda opción del uso de Gemini dentro de Google Colab es que te apoye en la generación de código que te permita visualizar mejor los resultados en el mapa. Para hacer uso de esta funcionalidad, puedes hacer clic en la opción de **“Generar con IA”** en la celda de código.

Interpretación asistida por IA

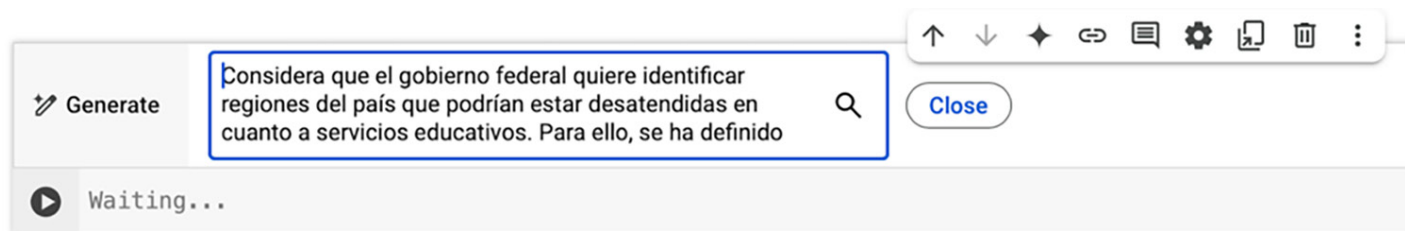


[] Start coding or **generate with AI.**

Figura 16. Botón Generar con IA.

Esto abre un campo en dónde puedes escribir un *prompt* que le explique a la IA lo que quieres lograr:

Interpretación asistida por IA



Generate

Considera que el gobierno federal quiere identificar regiones del país que podrían estar desatendidas en cuanto a servicios educativos. Para ello, se ha definido

Close

Waiting...

Figura 17. Campo para escribir el *prompt*.

Vamos a intentar con el siguiente *prompt*:

“Considera que el gobierno federal quiere identificar regiones del país que podrían estar desatendidas en cuanto a servicios educativos. Para ello, se ha definido una zona de influencia alrededor de cada capital estatal (por ejemplo, un radio de 50 km). Se busca determinar qué estados tienen la menor cobertura de estas zonas y, por lo tanto, podrían requerir atención prioritaria. El mapa generado muestra este resultado, pero me gustaría que la visualización me permitiera apreciar mejor qué zonas requieren más atención y en cuáles sería necesario agregar más centros educativos. ¿Cómo podría hacerlo?”

Es importante notar que la IA se puede equivocar. En este caso, el resultado que nos arroja muestra un error. Aprovechamos este error para mostrarte una última manera de interactuar con la herramienta de Gemini. En la parte inferior de la celda de código aparece el error y un botón que dice **“Explicar error”**.

```
-----
KeyError                                Traceback (most recent call last)
<ipython-input-6-8b3af717cae5> in <cell line: 0>()
    15
    16 # Agrupar por estado y sumar las áreas de cobertura
--> 17 cobertura_por_estado = estados_con_cobertura.groupby('CVE_ENT')['Area_Cobertura'].sum().reset_index()
    18
    19 # Unir la información de cobertura al GeoDataFrame de estados

-----
2 frames
/usr/local/lib/python3.11/dist-packages/pandas/core/groupby/grouper.py in get_grouper(obj, key, axis, level, sort, observed, validate, dropna)
    1041     in_axis, level, gpr = False, gpr, None
    1042     else:
-> 1043         raise KeyError(gpr)
    1044     elif isinstance(gpr, Grouper) and gpr.key is not None:
    1045         # Add key to exclusions

KeyError: 'CVE_ENT'
```

Next steps: [Explain error](#)

Figura 18. Botón explicar el error.

Si presionas este botón se va a abrir nuevamente el menú lateral derecho en donde la IA intentará ayudarte a comprender el error y solucionarlo. Nuevamente, ten cuidado con los resultados que podría arrojar, siempre es mejor verificarlos.

KeyError X

✦ **Hypothesis**

The error `KeyError: 'CVE_ENT'` arises because the code is trying to group the `estados_con_cobertura` GeoDataFrame by the column `'CVE_ENT'`, but this column does not exist in the GeoDataFrame. The `estados_con_cobertura` GeoDataFrame is created by intersecting the `estados` and `zonas_influencia` GeoDataFrames, and it is possible that the `'CVE_ENT'` column, which is present in the `estados` GeoDataFrame, is not included in the resulting `estados_con_cobertura` GeoDataFrame after the intersection operation.

Suggested Changes

```
# prompt: Considera que el gobierno federal quiere ide
from google.colab import drive
import pandas as pd
```

0 / 2000

Gemini can make mistakes so double-check responses and use code with caution. [Learn more](#)

Figura 19. Cuadro de ayuda de Gemini.

Después de corregir el código, el resultado generado es el siguiente:

```
from google.colab import drive
import pandas as pd
import geopandas as gpd
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors

# ... (código previo sin cambios) ...

# **Paso 5: Visualizar resultados con mejora para
# resaltar zonas desatendidas**

# Calcular el área de cobertura para cada estado
estados_con_cobertura['Area_Cobertura'] = estados_con_
cobertura.geometry.area
# Agrupar por estado y sumar las áreas de cobertura
cobertura_por_estado = estados_con_cobertura.
groupby('ISO3166-2')['Area_Cobertura'].sum().reset_
index()

# Unir la información de cobertura al GeoDataFrame de
# estados
estados = estados.merge(cobertura_por_estado,
on='ISO3166-2', how='left')
```

```

# Normalizar el área de cobertura para usarla en la
coloración
estados['Area_Cobertura_Normalizada'] = (estados['Area_
Cobertura'] - estados['Area_Cobertura'].min()) /
(estados['Area_Cobertura'].max() - estados['Area_
Cobertura'].min())

# Crear un mapa de colores divergente para resaltar las
diferencias
cmap = plt.cm.get_cmap('RdYlGn') # Rojo para baja
cobertura, verde para alta cobertura

fig, ax = plt.subplots(figsize=(12, 8))

# Graficar los estados con color según el nivel de
cobertura
estados.plot(column='Area_Cobertura_Normalizada',
cmap=cmap, linewidth=0.8, ax=ax, edgecolor='0.8',
legend=True, legend_kwds={'label': "Cobertura
Normalizada"})

# Agregar las zonas de influencia y la cobertura
zonas_influencia.plot(ax=ax, color='skyblue', alpha=0.3)
estados_con_cobertura.plot(ax=ax, color='green',
alpha=0.1) # Baja opacidad para que se vea el color de
los estados

plt.title("Cobertura educativa estimada por proximidad a
capitales")
plt.axis('off') # Ocultar los ejes
plt.show()

```

Asimismo, el mapa resultante es el siguiente:

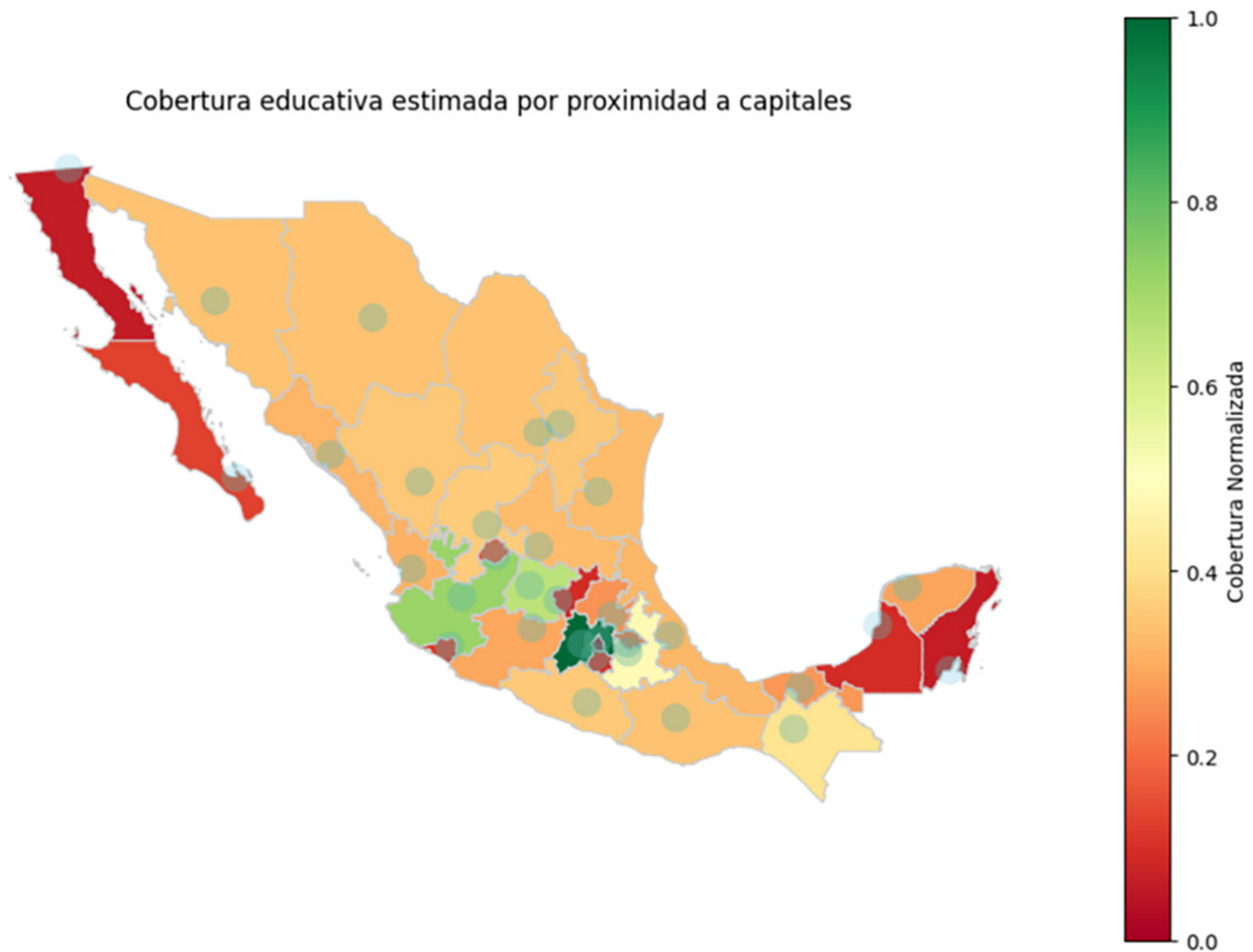


Figura 20. Mapa resultante del apoyo de la IA.

Como puedes ver, gracias al apoyo de la IA es más fácil apreciar en el segundo mapa las zonas que requieren atención, permitiendo identificar de forma más clara las áreas prioritarias para la toma de decisiones.

Mejores prácticas

Cuando trabajes con IA para interpretar información geoespacial, considera las siguientes buenas prácticas para formular mejores *prompts* y obtener resultados más precisos:

- **Sé claro y específico:** Indica de forma precisa qué datos quieres analizar, qué relaciones deseas encontrar, y qué tipo de visualización o respuesta esperas.
- **Proporciona contexto suficiente:** Explica brevemente de dónde provienen los datos, qué representan y cuál es el objetivo del análisis o interpretación.
- **Define el formato de la respuesta:** Especifica si deseas un resumen, una lista, una tabla, un análisis visual o un comentario interpretativo.
- **Incluye parámetros o restricciones relevantes:** Menciona aspectos como rangos de distancia, tipos de geometría o criterios de agrupación si son importantes para el análisis.
- **Evita ambigüedades:** Revisa tu *prompt* para asegurarte de que no haya términos confusos o instrucciones que puedan interpretarse de múltiples maneras.
- **Solicita ejemplos si es necesario:** Puedes pedir a la IA que incluya ejemplos o fragmentos de código para facilitar la aplicación práctica de los resultados.
- **Reformula y mejora tu *prompt*:** Si el primer resultado no es exactamente lo que buscabas, ajusta tu solicitud y vuelve a intentarlo, afinando detalles en cada iteración.

Aplicando estas recomendaciones, podrás aprovechar mejor las capacidades de la IA para enriquecer tus análisis geoespaciales y lograr visualizaciones más útiles y efectivas.

Toma en cuenta los siguientes puntos a modo de resumen de esta lección:

- **Los mapas interactivos permiten una exploración más profunda de los datos geoespaciales.** A través de herramientas como **Folium** y el método `explore()` de **GeoPandas**, puedes crear visualizaciones dinámicas que facilitan la comprensión de patrones y relaciones espaciales.
- **La inteligencia artificial puede asistir en la interpretación de mapas y resultados geoespaciales.** Utilizando asistentes de IA como **Gemini** en **Google Colab**, puedes automatizar el análisis, identificar zonas críticas y generar conclusiones preliminares de forma más rápida y precisa.
- **La claridad en la formulación de *prompts* es fundamental para obtener buenos resultados con IA.** Sé específico, proporciona contexto, define el tipo de respuesta que esperas y revisa tus instrucciones para mejorar progresivamente los resultados.
- **El sistema de referencia de coordenadas (CRS) debe ser consistente al trabajar con GeoDataFrames.** Antes de realizar operaciones geoespaciales o visualizaciones, asegúrate de que todos los datos trabajen bajo el mismo CRS para evitar errores en las interpretaciones espaciales.
- **Las operaciones de fusión espacial enriquecen la información visualizada.** Técnicas como `overlay`, `sjoin` y `sjoin_nearest` permiten combinar capas de información y facilitar análisis complejos, como la identificación de zonas de cobertura o regiones prioritarias.
- **La combinación de visualización inteligente + análisis asistido genera resultados más potentes.** No basta con graficar los datos: al apoyarte en herramientas de interpretación, mejoras la calidad del análisis y produces visualizaciones más útiles para la toma de decisiones.

Antes de terminar...

A lo largo de este tema, exploraste cómo la geovisualización puede ir más allá de los mapas estáticos para convertirse en una herramienta interactiva y, además, cómo la inteligencia artificial puede ser tu aliada en el análisis y la interpretación de datos geoespaciales.

Aprendiste a utilizar bibliotecas como **Folium** y el método `explore()` de **Geo-Pandas** para crear mapas dinámicos que permiten al usuario interactuar directamente con la información. También viste cómo integrar la asistencia de herramientas como Gemini de Google dentro de Google Colab para detectar patrones, generar hipótesis o visualizar áreas prioritarias de forma más eficiente.

Estos avances no sólo mejoran tu capacidad técnica, sino que también te ofrecen nuevas maneras de abordar problemas reales: ya sea optimizando recursos, mejorando la planificación territorial o facilitando la toma de decisiones basadas en datos. Hoy más que nunca, en un mundo lleno de información compleja, saber interpretar y comunicar datos geoespaciales de manera inteligente es una competencia valiosa, tanto a nivel personal como profesional.

Te invitamos a reflexionar sobre lo siguiente:

En tu vida profesional:

- ¿Cómo podrías integrar mapas interactivos o herramientas de interpretación asistida por IA en tus proyectos actuales?
- ¿Qué impacto tendría en la calidad de tus análisis o presentaciones?

En tu vida personal:

- ¿Cómo te cambiaría la perspectiva poder analizar de manera más dinámica y profunda tu entorno, tu ciudad o incluso los lugares que planeas visitar o estudiar?

Recuerda: no se trata sólo de aprender una herramienta, sino de transformar tu manera de observar, analizar y actuar sobre el mundo que te rodea.

Material adicional

Fine, J.; Kolla, M. & Soloduchko, I. (03 de marzo de 2025). *Data Science Agent in Colab: The future of data analysis with Gemini*. [Agente de Ciencia de Datos en Colab: El futuro del análisis de datos con Gemini.]. Google for Developers. Recuperado el 06 de mayo de 2025 de: <https://developers.googleblog.com/en/data-science-agent-in-colab-with-gemini/>

Folium. (s.f.) *Folium*. Python-visualization. Recuperado el 06 de mayo de 2025 de: <https://python-visualization.github.io/folium/latest/>

Folium. (s.f.) *Map*. [Mapa]. Python-visualization. Recuperado el 06 de mayo de 2025 de: https://python-visualization.github.io/folium/latest/user_guide/map.html

Support Google. (s.f.) Ayuda legal. Recuperado el 06 de mayo de 2025 de: <https://support.google.com/legal/answer/13505487>

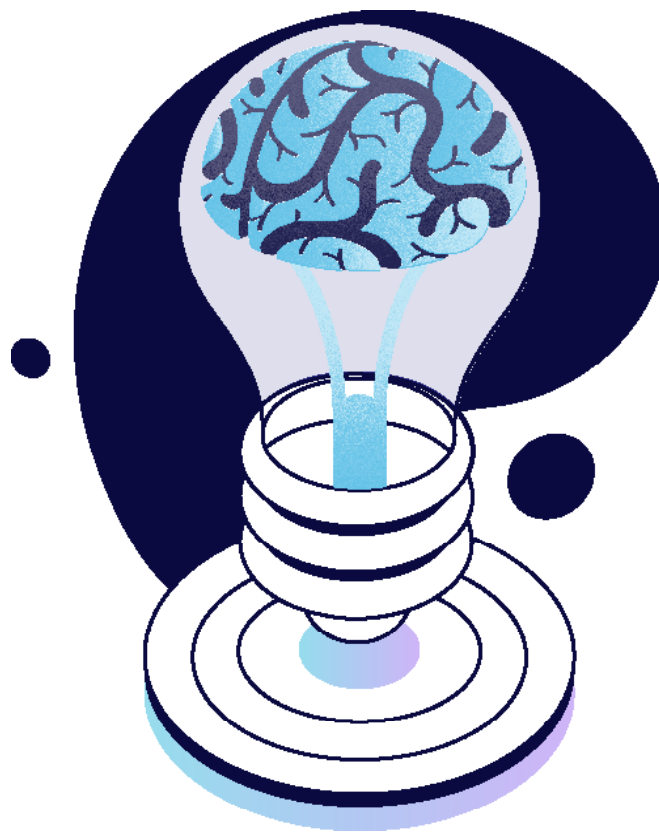
Yunta, E. (s.f.) 8 Aplicaciones de la inteligencia artificial en los SIG. MappingGis. Recuperado el 06 de mayo de 2025 de: <https://mappinggis.com/2024/11/aplicaciones-de-la-inteligencia-artificial-en-los-sig/>

Nota: Los enlaces a las páginas web incluidos en este material adicional fueron consultados y verificados por última vez el 06 de mayo de 2025 y su disponibilidad depende del autor.

Glosario

- **Análisis espacial:** Proceso de examinar la ubicación, las relaciones y los patrones de datos geográficos para responder preguntas o resolver problemas.
- **`explore()`:** Método de **GeoPandas** que genera una visualización interactiva de un `GeoDataFrame` utilizando tecnología de mapas en el navegador.
- **Folium:** Biblioteca de **Python** utilizada para crear mapas interactivos de forma sencilla, basada en la biblioteca `Leaflet.js`.
- **GeoPandas:** Extensión de `pandas` que permite trabajar con datos geoespaciales (puntos, líneas, polígonos) dentro de estructuras tipo `DataFrame`.
- **Geovisualización:** Representación visual de datos geoespaciales que permite explorar, analizar y comunicar información basada en ubicación.
- **Google Colab:** Plataforma en línea que permite programar en **Python** de manera colaborativa, con acceso integrado a recursos de inteligencia artificial.
- **Inteligencia Artificial (IA):** Conjunto de tecnologías que permiten a las máquinas simular capacidades humanas como el razonamiento, el aprendizaje y la interpretación de datos.
- **Interpretación asistida:** Análisis de resultados facilitado por herramientas de IA, que ayudan a identificar patrones, anomalías o tendencias a partir de datos geoespaciales.
- **Mapa interactivo:** Representación dinámica que permite al usuario interactuar con los datos mediante acciones como hacer *zoom*, seleccionar elementos o desplegar información adicional.

- **Prompts:** Instrucciones o preguntas claras que se le dan a un modelo de IA para guiar su respuesta de manera precisa y relevante.
- **Sistema de Referencia de Coordenadas (CRS):** Sistema que define cómo se interpretan las coordenadas espaciales en un mapa, garantizando la correcta representación geográfica.
- **Unión espacial (*Spatial Join*):** Técnica para fusionar datos de dos conjuntos geoespaciales basándose en relaciones espaciales como intersección o proximidad.



Bibliografía

Alteryx. (s.f.) *Análisis espacial*. Alteryx. Recuperado el 05 de mayo de 2025 de: <https://www.alteryx.com/es/glossary/spatial-analysis>

Biodiversidad mexicana. (18 de octubre de 2023). *Geovisualización*. Biodiversidad mexicana. Recuperado el 05 de mayo de 2025 de: <https://www.biodiversidad.gob.mx/region/geoviz>

Charlotte. (19 de septiembre de 2024). *AI Helpful Tips: Creating Effective Prompts*. [Consejos útiles de IA: cómo crear prompts efectivos.] Charlotte. Recuperado el 06 de mayo de 2025 de: <https://oneit.charlotte.edu/2024/09/19/ai-helpful-tips-creating-effective-prompts/>

CRSEU (s.f.) *Definition of a Coordinate Reference System (CRS)*. [Definición de un Sistema de Referencia de Coordenadas (CRS).] CRSEU. Recuperado el 06 de mayo de 2025 de: <https://www.crs-geo.eu/definition-crs.htm>

Folium. (s.f.) *Map*. [Mapa]. Recuperado el 06 de mayo de 2025 de: https://python-visualization.github.io/folium/latest/user_guide/map.html

Folium. (s.f.) *Folium*. Python-visualization. Recuperado el 06 de mayo de 2025 de: <https://python-visualization.github.io/folium/latest/>

Gemini. (s.f.). *Google*. Recuperado el 06 de mayo de 2025: <https://gemini.google.com/>

GeoPandas. (s.f.) *GeoPandas 1.0.1*. GeoPandas. Recuperado el 06 de mayo de 2025 de: <https://geopandas.org/en/stable/>

GeoPandas. (s.f.) *geopandas.GeoDataFrame.explore*. GeoPandas. Recuperado el 06 de mayo de 2025 de: <https://geopandas.org/en/stable/docs/reference/api/geopandas.GeoDataFrame.explore.html>

GeoPandas. (s.f.) *Interactive mapping*. [Mapeo interactivo]. GeoPandas. Recuperado el 06 de mayo de 2025 de: https://geopandas.org/en/stable/docs/user_guide/interactive_mapping.html

GeoPandas. (s.f.) *Plotting with Folium*. [Visualización con Folium]. GeoPandas. Recuperado el 06 de mayo de 2025 de: https://geopandas.org/en/stable/gallery/plotting_with_folium.html

GeoPandas. (s.f.) *Spatial Joins*. [Uniones espaciales]. GeoPandas. Recuperado el 06 de mayo de 2025 de: https://geopandas.org/en/stable/gallery/spatial_joins.html

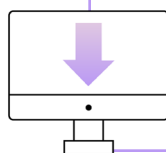
Google Cloud. (s.f.) *¿Qué es la inteligencia artificial o IA?*. Google Cloud. Recuperado el 06 de mayo de 2025 de: <https://cloud.google.com/learn/what-is-artificial-intelligence?hl=es-419>

Google Colab. (s.f.) *Google*. Recuperado el 05 de mayo de 2025: <https://colab.research.google.com/>

IA geoespacial. (s.f.) *Inteligencia artificial en la tecnología SIG*. Esri. Recuperado el 06 de mayo de 2025 de: <https://www.esri.com/es-es/geospatial-artificial-intelligence/overview>

Meier, K. (s.f.) *10 tools to create interactive maps*. [10 herramientas para crear mapas interactivos]. Shorthand. Recuperado el 06 de mayo de 2025 de: <https://shorthand.com/the-craft/tools-to-create-interactive-maps/index.html>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 06 de mayo de 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.